



The work is the interface.

A UI and UX proposal for UDAX, the IDE for user, developer, and agentic experience. Issues become a living graph on one canvas, and people and agents build software on it together.

Design proposal, September 2026

The bottleneck moved from typing to deciding.

Agents now write most of the diff. The scarce work is deciding what to build, splitting it well, steering it, and checking what comes back.

Today that work is spread across four tools that don't share context. People carry it between them by hand.

Today

Tracker

Status updated by hand, days late

copy link

Agent chat

Context re-explained every session

-- paste --

Editor

One repo per window

-- push --

Pull request

"Which issue is this for?"

In UDAX

CHK-32 **Wallet buttons on web checkout**

Wren is editing OrderSummary.tsx

Intent and criteria

Apple Pay and Google Pay on web checkout. Falls back to the card form.

Repositories

web ds

chk-32/wallet-buttons

Pull requests

web#482

Evidence

2 of 3 criteria verified

One object carries the intent, the code, the agent, the review, and the proof. Nobody copies anything.

One graph, three kinds of users.

U, D, and A each read the same issue differently. UDAX renders the same node for each of them, from the same source of truth.

U **User**
Product leads, founders, designers

Say what should exist. See where it stands without asking anyone.

D **Developer**
Engineers and tech leads

Shape the work, steer agents mid-flight, and own the quality of what merges.

A **Agent**
Coding, planning, and review agents

Get exact context, act inside clear limits, and report back in a structure people can check.

🔔 Wallet buttons on web checkout

TR Theo owns this. Wren, an agent, is building it.

Apple Pay and Google Pay work. The card fallback is next, then review.

- Apple Pay on Safari
- Google Pay on Chrome
- Card fallback

2 of 3 outcomes verified. Expected in review today.

🔔 CHK-32

Ask first

TR Owner: Theo from CHK-12

W Assignee: Wren

web chk-32/wallet-buttons

🔗 web#482

ds chk-32/wallet-buttons

2 files, no PR

● Kit (CHK-41) is also editing OrderSummary.tsx

W Wren 6 of 9 checks, 12 min in, budget 45 min

```
issue: CHK-32
intent: Add Apple Pay and Google Pay
       buttons to web checkout
inherits: [CHK-1, CHK-12]
owner: theo.ramos # questions go here
agent: wren v7    # sonnet-5.5, high
accept:
  - Apple Pay shows on Safari # e2e
  - Google Pay shows on Chrome # e2e
  - Falls back to card form   # unit
repos:
  web: write      # chk-32/wallet-buttons
  ds:  write      # src/Button/**
contracts: [CHK-31 POST /wallet/tokens]
autonomy: ask-before-merge
budget: 45m
```

Six rules the design follows.

The issue is the unit of work.

Branches, worktrees, PRs, and agent runs hang off issues. Never the other way around.

Agents are teammates on the canvas.

They work where you can see them and talk in the issue's conversation. A person always owns the outcome.

Zoom is navigation.

One canvas runs from the portfolio to a line of code. No tabs to lose your place in.

Proposals are ghosts until accepted.

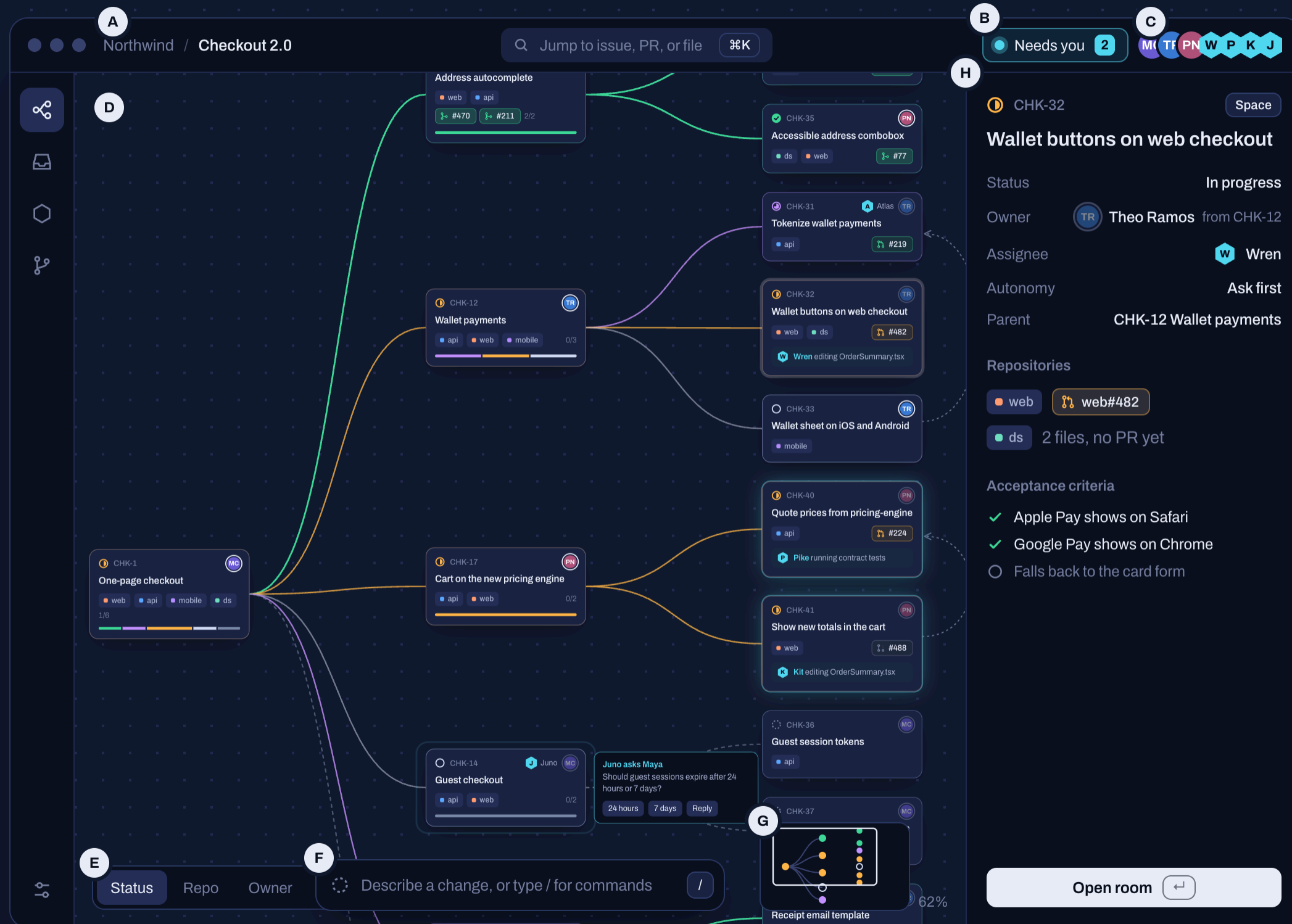
Anything an agent suggests is visibly unconfirmed, free to drop, and never runs on its own.

Status is readable from any height.

Color and glyph survive every zoom level. You can read a project's health at 8%.

Approve by evidence.

Review starts with “does it meet the criteria?” and only then “is the diff good?”

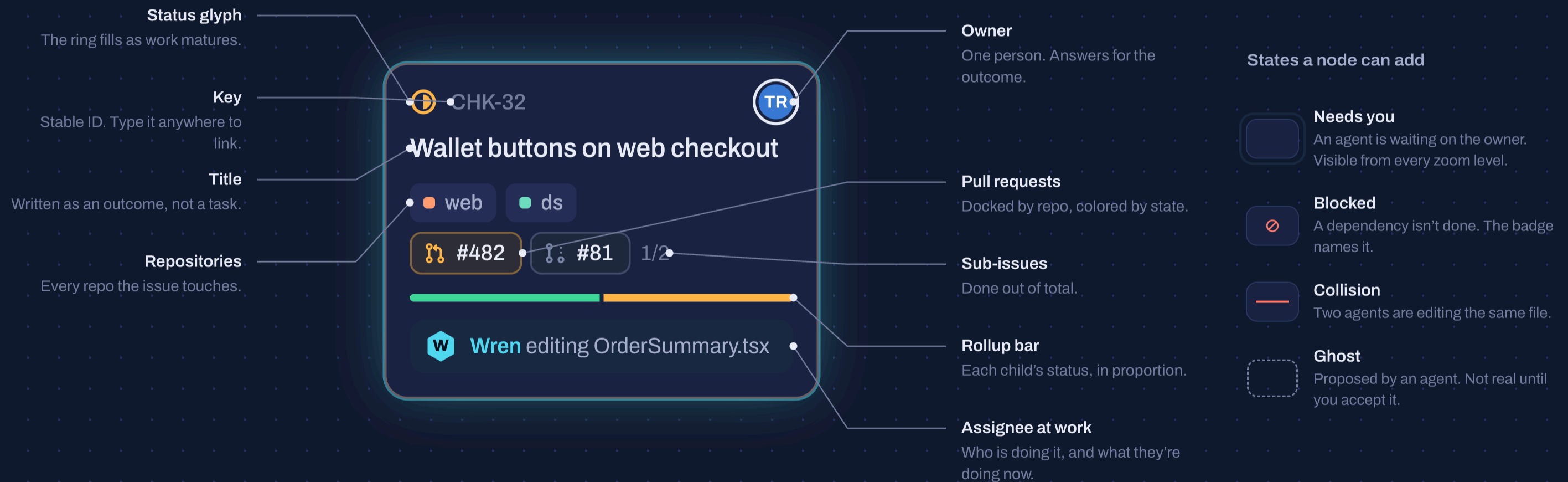


The canvas.

- A** Project path. Where you are in the portfolio. Click to rise.
- B** Needs you. Questions and approvals for issues you own, across every project.
- C** Presence. People and agents active here right now.
- D** Issue graph. Sub-issues branch to the right. Color and glyph show status.
- E** Lenses. Recolor the same graph to answer a different question.
- F** Intent bar. Describe a change in plain words, or run a command.
- G** Minimap. Where the viewport sits, and how far you've zoomed.
- H** Peek. Repos, PRs, and criteria for the selected issue without leaving the canvas.

Anatomy of an issue.

A node is a card you can read at arm's length. Everything on it is live: status, agent, branches, and checks update without a refresh.



One owner. Any number of hands.

Agents can do the work, but they can't answer for it. So UDAX separates the two.



Owner

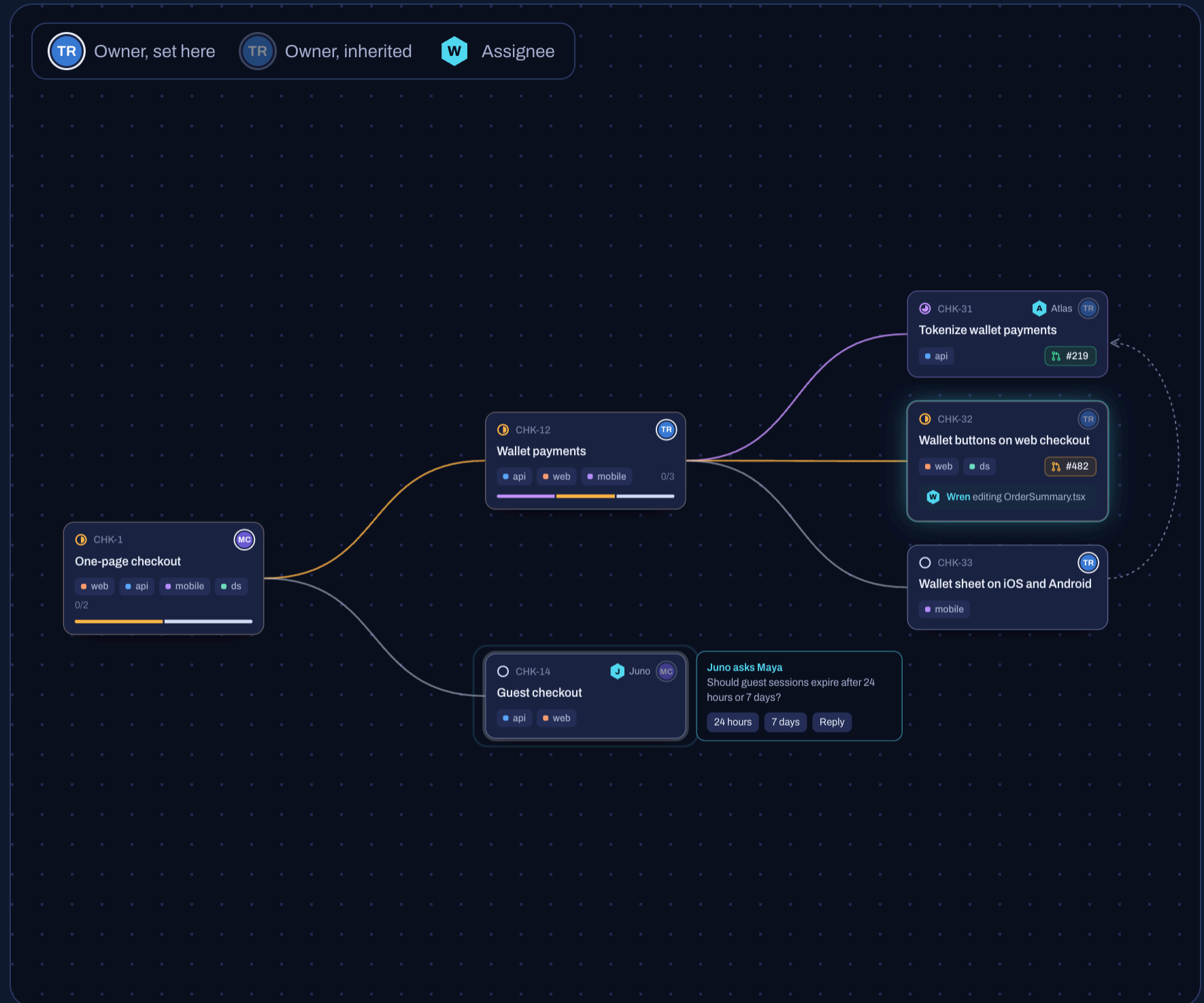
Answers for the outcome. Exactly one, always a person. Inherited from the parent unless set.



Assignees

Do the work. People or agents, as many as needed, handed off freely.

- **Questions go to the owner.** If the owner doesn't answer, they go to the parent's owner.
- **The owner accepts the outcome.** Code owners approve the diff.
- **Autopilot needs a person as owner.** An owner can't be removed while agents are working.



Five statuses. The ring fills as work matures.

Status is carried by shape as well as color, so it reads for everyone. It moves on real events, not on someone remembering to drag a card.



Backlog

Fog #7C87A8

Captured, not committed. Cheap to create, cheap to drop.

- 1 Created from the intent bar, an accepted ghost, or an import.



To do

Chalk #D6DEF5

Committed and ready. Has an owner and no open blockers.

- 2 Pulled into the current cycle, or its last blocker merges.



In progress

Saffron #FFB547

A person or an agent is working on it in a real branch.

- 3 The first branch is created or an agent run starts.



In review

Orchid #C792FF

Waiting for a human decision. The work is done as far as the agent can tell.

- 4 Every linked PR is ready and its checks pass.



Done

Jade #3FD99A

Merged and verified. Every criterion has evidence attached.

- 5 Every PR is merged and every criterion is checked.

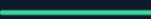
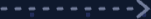
Parents roll up. A parent can't be Done while a child is open. Its rollup bar shows why.

Overrides are allowed. Moving status by hand asks for a reason and leaves a marker on the node.

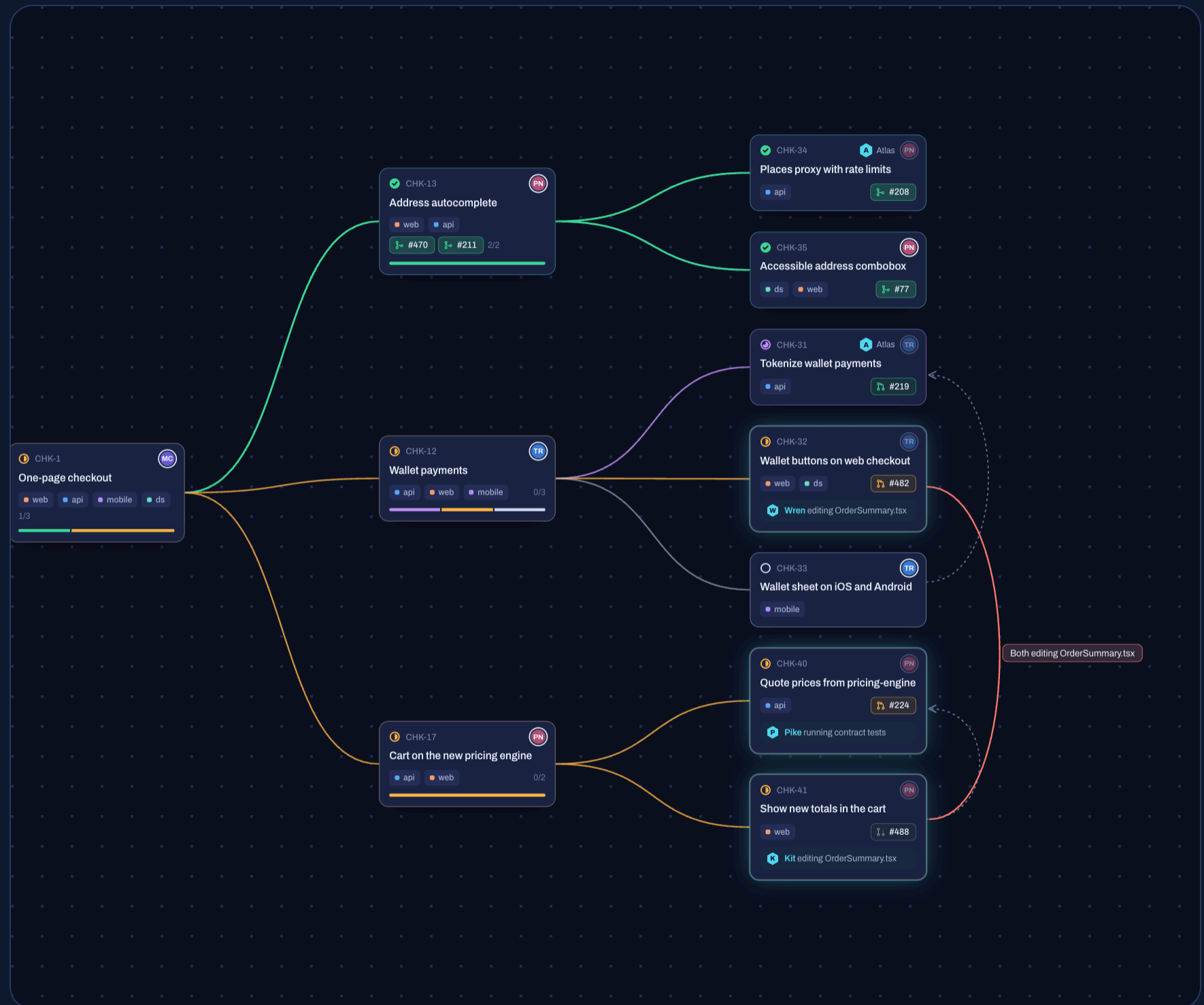
Backwards is normal. A failed check sends In review back to In progress, and the agent is told why.

Branches show structure. Color shows progress.

As work completes, branches light up from the leaves back to the root. A finished project looks finished from across the room.

-  **Sub-issue.** Solid, tinted by the child's status.
-  **Sub-issue in backlog.** Dashed until someone commits to it.
-  **Depends on.** Dotted, the arrow points at the blocker.
-  **Collision.** Two agents editing the same file.

 Show what CHK-33 is waiting on



Zoom is navigation.

One canvas runs from the whole portfolio to a single line of code. Each altitude shows only what you can act on from there.

8% **Portfolio.** Projects as clusters of dots. Only status and needs-you beacons survive.

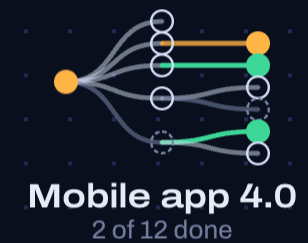
25% **Project.** Issues shrink to pills. Good for planning and spotting stuck branches.

60% **Board.** Full cards with owners, assignees, repos, and PRs.

100% **Issue.** Cards open in place: branches, PR checks, criteria, blockers.

↵ **Room.** Step inside to see code, diffs, previews, and the agent's log.

Scroll or pinch to move between altitudes. ↵ enters a room, Esc rises one level.



Branching out. Agents propose, you prune.

Press **s** on any issue and an agent drafts sub-issues as ghosts. Ghosts cost nothing, run nothing, and vanish unless you keep them.

- **Every ghost explains itself:** why it exists, which repos it touches, and a rough size.
- **Keep or drop in one key.** **↩** accepts, **E** edits, **🗑** drops, **↩↩** keeps all.
- **Dropped ghosts are remembered** as out of scope for this project, so they don't come back.

S Ask Juno to plan

Reset

○ CHK-14

Juno

MC

Guest checkout

api

web

Proposed

About 2h

Guest session tokens

session.go assumes every cart has a user ID.

api

↩ Accept

E Edit

🗑 Drop

Proposed

About 1h

Collect and verify guest email

Receipts and order lookup need an address before payment.

web

↩ Accept

E Edit

🗑 Drop

Proposed

About 3h

Claim an account after purchase

Links guest orders to an account created later.

api

web

↩ Accept

E Edit

🗑 Drop

Proposed

About 1d

Move sessions to Redis

Would simplify expiry. Not needed at guest volume.

infra

↩ Accept

E Edit

🗑 Drop

One issue, many repositories.

Features cross service boundaries. An issue owns one branch name, created in every repo it touches, each in its own worktree.

- **Repos are inherited.** Sub-issues start with the parent's repos and narrow them down.
- **Worktrees are automatic.** Created when work starts and removed after merge, so parallel agents never share a checkout.
- **Contracts are checked across repos.** A type change in api flags every PR that depends on it.

CHK-12 **Wallet payments**

chk-12/wallet-payments Same branch name in every repository

Repository	Progress	Worktree	Pull request	Sub-issue
api		Ready, 14 files	api#219	CHK-31
web		Ready, 9 files	web#482	CHK-32
ds		Ready, 2 files	No PR yet	CHK-32
mobile		Not created	Waits on CHK-31	CHK-33

+ Add repository R



Contract changed in api#219

POST /wallet/tokens now returns expiresAt. web#482 reads the old shape. Wren has been sent the new type.

Open diff

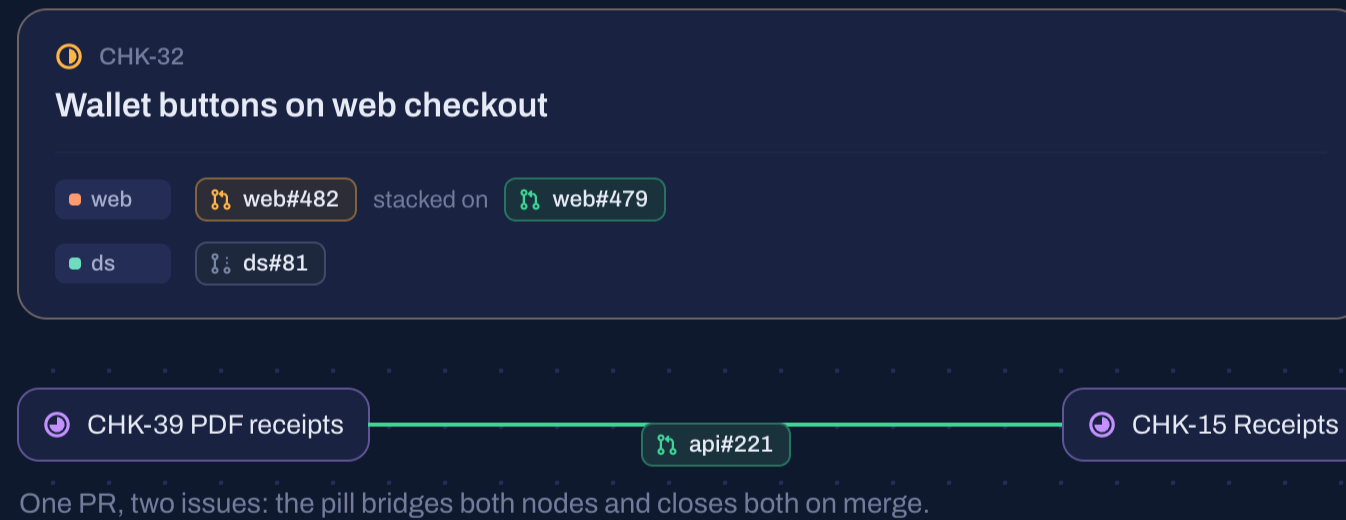
Pull requests dock onto issues.

An issue can hold many PRs across repos, and one PR can close several issues. Linking is automatic, and anything that slips through is easy to catch.

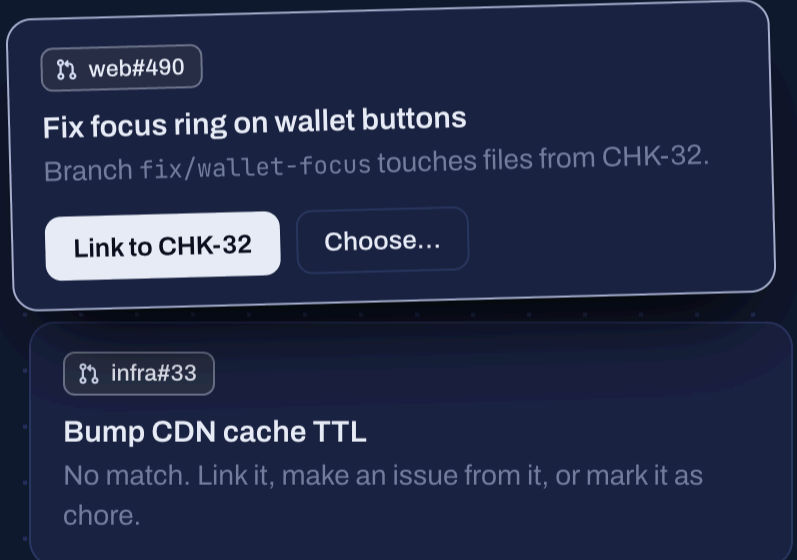
PR states reuse the status inks

 web#488	Draft
 web#491	Ready for review
 web#482	Checks running
 api#224	Checks failing
 ds#80	Changes requested
 api#219	Approved
 api#215	Merged

A dock, stacked by repo



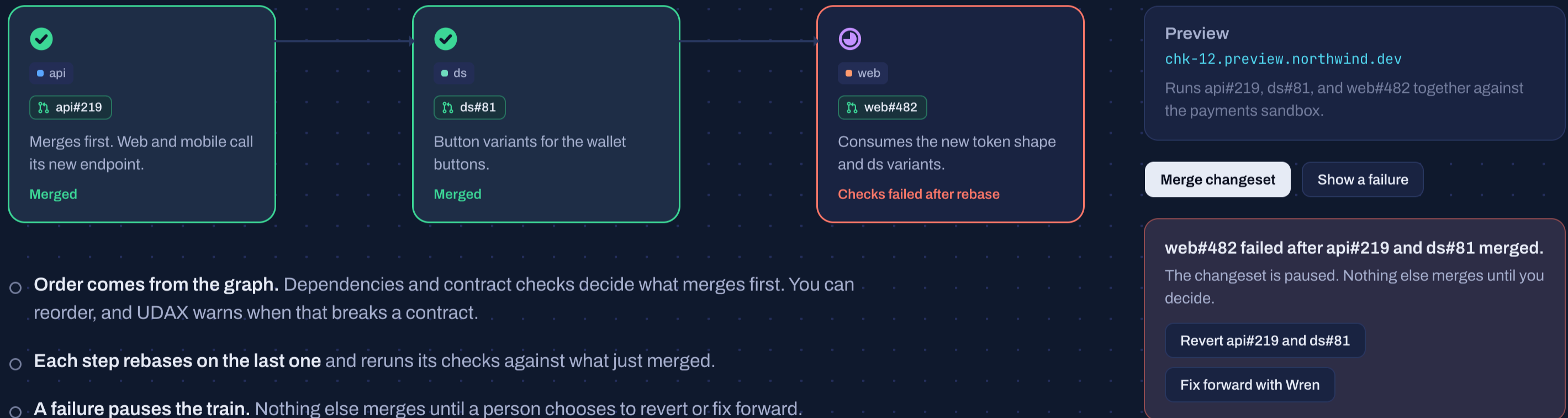
Loose ends



- PRs link from branch names, agent runs, or “Closes CHK-32”.
- Drag any loose PR onto a node to link it.

Multi-repo work ships as one changeset.

When an issue spans repos, its PRs merge together in dependency order. One preview runs every branch at once, so you test the feature, not the fragments.

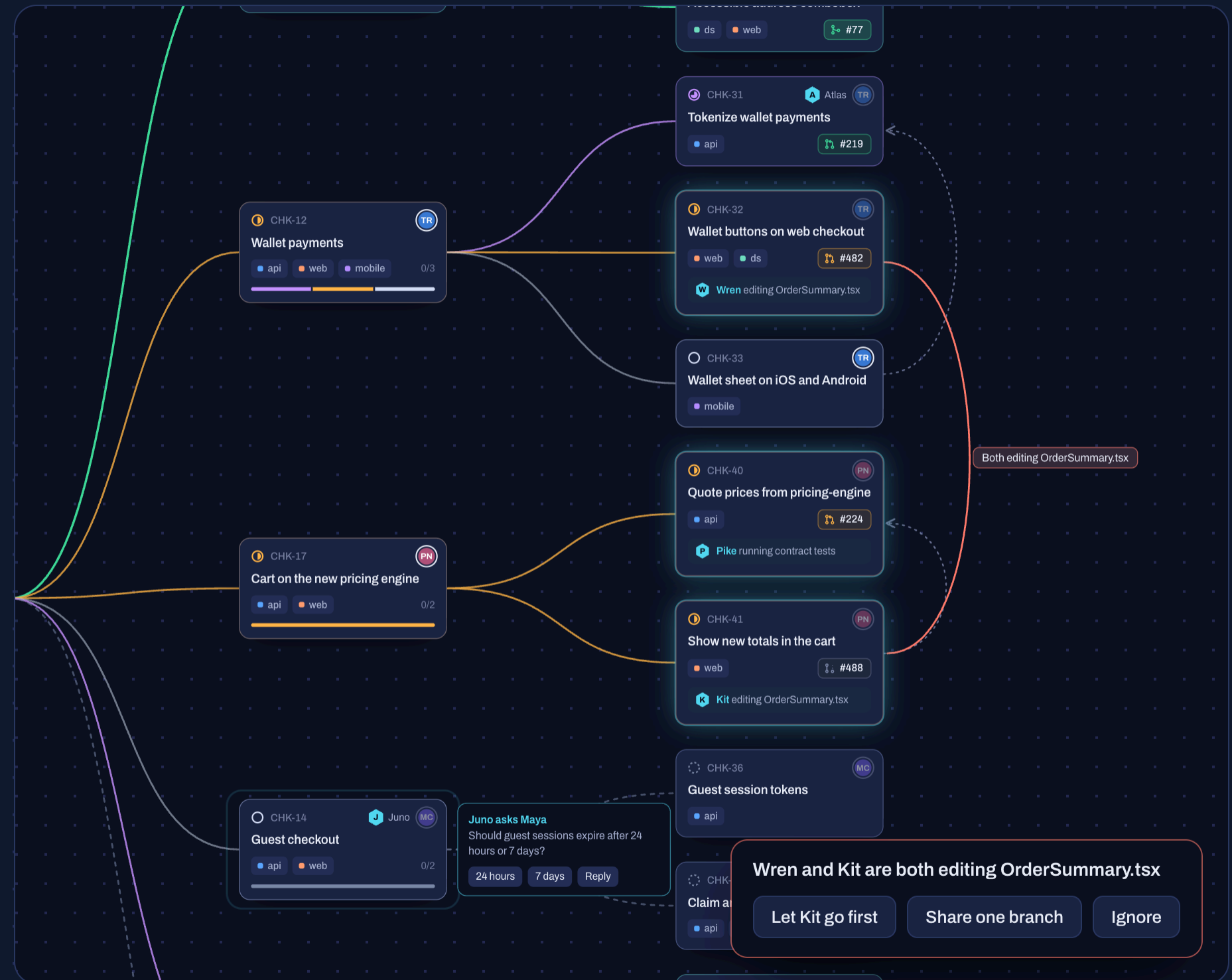


- **Order comes from the graph.** Dependencies and contract checks decide what merges first. You can reorder, and UDAX warns when that breaks a contract.
- **Each step rebases on the last one** and reruns its checks against what just merged.
- **A failure pauses the train.** Nothing else merges until a person chooses to revert or fix forward.

Agents work in the open.

No separate chat window. You see agents where they work, and you talk to them in the issue's conversation.

- **Live agents glow cyan** and narrate their current step on the node.
- **Needs you goes to the owner** as a beacon on the node, visible from every altitude. **Tab** jumps to the next one. Answer inline without opening anything.
- **Collisions show up before conflicts.** When two agents edit the same file, a coral link appears with one-click fixes.



Agents are teammates you configure.

Each agent is a role, a system prompt, a model, and an effort level. Edit it once and every issue it picks up gets the new version.

Northwind / Settings / Agents

New agent

W

Wren

Frontend builder

Sonnet 5.5

A

Atlas

Backend and payments

Opus 5.5

P

Pike

Pricing and contracts

Sonnet 5.5

K

Kit

UI polish

Haiku 4.5

J

Juno

Planner

Opus 5.5

W

Wren

Frontend builder. Working on CHK-32.

Prompt v7

System prompt

1

You build frontend features for Northwind’s web checkout.

2

Use components from ds. Never hand-roll a button or an input.

3

Start by restating the acceptance criteria in a comment.

4

Ask `{{owner.name}}` before touching `{{protected_paths}}`.

5

If checks fail twice, stop and comment with the log.

6

Reply to every steering comment with what you changed.

Model

Claude Sonnet 5.5

Effort

Low

Medium

High

Max

High plans before it edits and checks its own work. Runs take longer and cost more.

Access

web

ds

api

Write

Write

Read only

Defaults

Autonomy

Ask first

Budget

45 min, \$6

Recent runs

Issue	Prompt	Effort	Result	Time
CHK-32 Wallet buttons	v7	High	Running	15 min
CHK-35 Address combobox	v6	Medium	Merged, 1 change asked	22 min
CHK-29 Promo code field	v6	Medium	Merged first try	9 min

Five agents, four working now.

v7 saved by Theo yesterday. Each run records the version it used.

History

Try on an issue

Save as v8

Match the agent to the work.

Assign from the node. UDAX suggests an agent from the issue's repos and history, and you can change the model or effort for one run.

- Low** Copy changes, renames, small fixes.
- Medium** One file with clear criteria.
- High** Work across several files. The default.
- Max** Unclear bugs, contracts across repos, design calls.

Every run records its agent, prompt version, model, and effort, so you can compare runs and repeat the good ones.

The screenshot displays the UDAX interface. On the left, a workflow graph shows three nodes: 'CHK-14 Guest checkout' (with 'api' and 'web' tags), 'CHK-36 Guest session tokens' (with 'api' tag), and 'CHK-37 Claim an account after purchase' (with 'api' and 'web' tags). Dashed lines connect 'CHK-14' to both 'CHK-36' and 'CHK-37'. On the right, a panel titled 'Assign CHK-36 to...' shows suggested agents. The 'Suggested' section lists 'Atlas' (Backend, Built the tokens in CHK-31, Opus 5.5) with a 'Low' effort indicator. Below, the 'Agents' section lists 'Wren' (Frontend builder, Sonnet 5.5) and 'Pike' (Busy on CHK-40, Sonnet 5.5). The 'People' section lists 'Theo Ramos' (Owns CHK-12). At the bottom, a 'For this run only' section allows selecting the model (Claude Opus 5.5) and effort (Low, Medium, High, Max). A note states: 'Session tokens touch auth. Saved on the issue, not on Atlas.' The 'Assign Atlas' button is highlighted.

Assign CHK-36 to...

Suggested

Atlas
Backend. Built the tokens in CHK-31. Opus 5.5

Agents

Wren
Frontend builder. Sonnet 5.5

Pike
Busy on CHK-40. Sonnet 5.5

People

Theo Ramos
Owns CHK-12

For this run only

Model: Claude Opus 5.5

Effort: Low Medium High **Max**

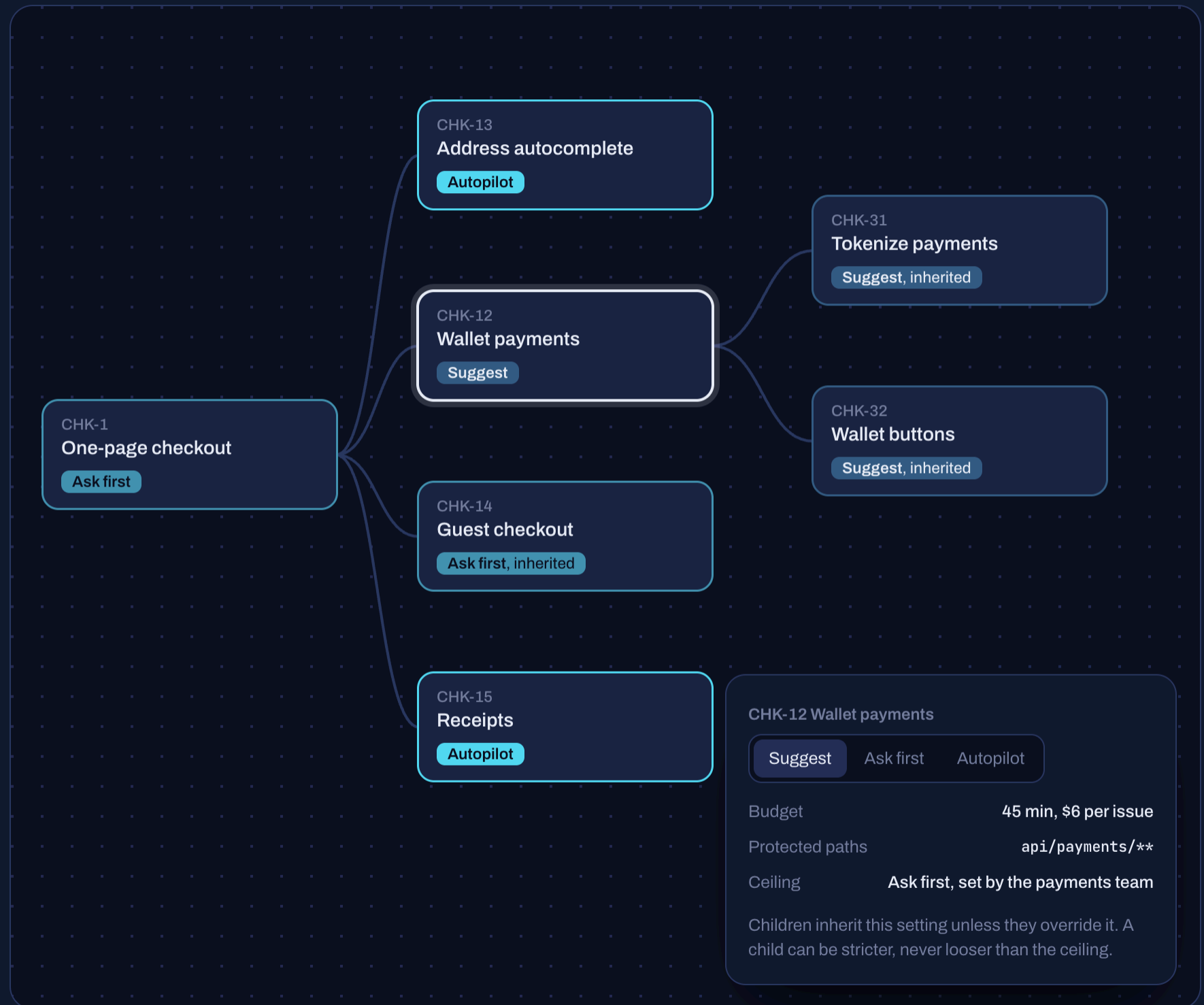
Session tokens touch auth. Saved on the issue, not on Atlas.

Assign Atlas

Autonomy is a dial, set per branch.

Trust isn't global. Low-risk work can run on autopilot while payments stays on a short leash, in the same project.

- ◆ **Suggest** Plans and drafts only. Proposes ghosts and diffs, changes nothing without you.
- ◆ **Ask first** Works in its own worktree and opens PRs. Asks before merging, going over budget, or touching protected paths.
- ◆ **Autopilot** Takes the issue to Done within budget. The owner gets a summary and can undo the changeset.



Enter an issue to work inside it.

The room is the issue's workspace. Criteria on the left, five views in the middle, repos and the agent on the right. It opens on the conversation.

● ● ● Checkout 2.0 / CHK-12 Wallet payments / **CHK-32 Wallet buttons on web checkout**

In progress

Suggest

Ask first

Autopilot

Esc

Canvas

Intent

Shoppers with a saved wallet can pay in one tap. Everyone else sees the card form exactly as today.

Acceptance criteria

✓ Apple Pay shows on Safari
e2e passed, 2 min ago

✓ Google Pay shows on Chrome
e2e passed, 2 min ago

○ Falls back to the card form
Test written, not run yet

Context

🕒 CHK-1 One-page checkout

🕒 CHK-12 Wallet payments

🕒 CHK-31 contract: POST /wallet/tokens

People

TR Theo Ramos owner, from CHK-12

W Wren assignee

Conversation Activity Code Preview Terminal

3 requests, 1 in progress

TR Theo Ramos Owner 09:05

@Wren use the sandbox merchant ID in previews, not production.

W Wren 09:06

Switched APPLE_MERCHANT_ID in .env.preview. Production is untouched.

🔑 a41c9e2

W Wren Asked Theo 09:11

Apple Pay needs a domain file at /.well-known/. Should it live in web, or does infra own that path?

TR Theo Ramos 09:12

Put it in web for now. Open an issue for infra to own it in production.

MC Maya Chen 09:16

📍 Preview, pin 1

The Apple Pay button is taller than the card field. Can they match?

W Wren is working on this ...

Steer

Note

Comment to steer Wren. Type @ to mention someone.

Comment ↵

Repositories

web

web#482

6 of 9

ds

2 files, no PR yet

Agent

W Wren

Frontend builder

Sonnet 5.5

High

Prompt v7

Run 2

15 min, \$0.84

Now

Editing OrderSummary.tsx

Pause

Take over

Steer with comments.

The conversation belongs to the issue, not to a chat window. A comment to an agent becomes a request with a status, and the agent replies in the thread with what it changed.

● ● ● Checkout 2.0 / CHK-12 Wallet payments / **CHK-32 Wallet buttons on web checkout**

In progress

TR W

Esc Canvas

ConversationActivityCodePreviewTerminal

Oldest first

@Wren use the sandbox merchant ID in previews, not production.

W Wren 09:06

Switched APPLE_MERCHANT_ID in .env.preview. Production is untouched.

a41c9e2

W Wren Asked Theo 09:11

Apple Pay needs a domain file at /.well-known/. Should it live in web, or does infra own that path?

Put it in web

Infra owns it

TR Theo Ramos 09:12

Put it in web for now. Open an issue for infra to own it in production.

Decision

W Wren 09:13

Added in web. I proposed CHK-44 under CHK-12 for infra. It's a ghost until you accept it.

3b7f010

Proposed: Serve the domain file from the edge

MC Maya Chen 09:16

Preview, pin 1

The Apple Pay button is taller than the card field. Can they match?

W Wren is working on this

Requests

✓ Use the sandbox merchant ID

Theo, done in a41c9e2

✓ Serve the domain file from web

Theo, done in 3b7f010

In progress Match the button heights

Maya, in progress

Decisions

The domain file lives in web until infra owns it.

Theo, 09:12

Decisions join the agent's context for this issue and every sub-issue under it.

Steer or note

Steer sends a comment to the assignee as a request. Note starts no work, but agents can still read it, like the rest of the conversation.

SteerNote

Comment to steer Wren. Type @ to mention someone, or select code, a pin, or output to anchor it.

Comment

Activity is the full record, grouped by run.

Every agent run, commit, check, and status change in one timeline. Tool calls fold into readable steps, and each run shows its model, effort, prompt, time, and cost.

Checkout 2.0 / CHK-12 Wallet payments / CHK-32 Wallet buttons on web checkout

In progress

TRW

EscCanvas

ConversationActivityCodePreviewTerminal

AllAgentsPeopleChecksStatus

Newest first

Wren, run 2

Sonnet 5.5

High

Prompt v7

Running15 min, \$0.84

09:14

Editing OrderSummary.tsx

09:13

Served the Apple Pay domain file from web

3b7f010

09:12

Ran unit tests

42 passed

09:09

Added wallet buttons and two ds variants

9c2e71b

+184 -6

readds/src/Button/Button.tsx

writeweb/src/checkout/WalletButtons.tsx+96

writeds/src/Button/variants.ts+41

runpnpm -C ds buildpassed in 8.2s

09:06

Replied to Theo and switched the merchant ID

a41c9e2

> Wren, run 1

Sonnet 5.5

Medium

Prompt v6

Stopped by Theo to raise the effort. 4 min, \$0.18

08:55

Moved to In progress when chk-32/wallet-buttons was created

So far

Runs2

Agent time19 min

Cost\$1.02

Commits3

Budget

Time19 of 45 min

Cost\$1.02 of \$6

At 80% of either budget, Wren asks Theo before it continues.

Compare runs

SteerNote

Comment to steer Wren. Type @ to mention someone, or select code, a pin, or output to anchor it.

Comment

Code shows what this issue changed.

One file tree across every repo the issue touches, with the agent's reasoning beside its edits. Select a line to comment on it, or edit it yourself.

Checkout 2.0 / CHK-12 Wallet payments / **CHK-32 Wallet buttons on web checkout**

In progress

TR W

Esc Canvas

Conversation Activity **Code** Preview Terminal

7 files, +281 -10

Changed All files

web src/checkout

WalletButtons.tsx **New** +96

OrderSummary.tsx +12 -3

CardFallback.tsx **New** +31

.env.preview +1 -1

.well-known/apple-... **New** +1

ds src/Button

variants.ts +41

Button.stories.tsx +18

WalletButtons.tsx OrderSummary.tsx

Changes Whole file

```
12 export function WalletButtons({ total, onToken }: Props) {
13   const wallets = useAvailableWallets();
14 + if (!wallets.length) return <CardFallback />;

15   return (
16     <Stack gap="2">
17 +   {wallets.includes('apple') && (
18 +     <ApplePayButton amount={total} onToken={onToken} />
19 +   )}
20 +   {wallets.includes('google') && (
21 +     <GooglePayButton amount={total} onToken={onToken} size="md" />
22 +   )}
```

W Returning early means the card form never renders twice. Criterion 3.

TR Theo Could the fallback reuse CardForm instead of a new component?

W Wren Yes. I'll swap it in after the button heights.

Queued

TR Theo is editing line 21. Wren holds off on this file and picks up the change when Theo stops.

Steer Note

Comment to steer Wren. Type @ to mention someone, or select code, a pin, or output to anchor it.

Comment

Preview the branch, not a screenshot.

A live build of every branch in the changeset. Switch browser and device, compare with main, and pin comments on what you see.

Checkout 2.0 / CHK-12 Wallet payments / CHK-32 Wallet buttons on web checkout

In progress

TRW

EscCanvas

ConversationActivityCodePreviewTerminal

chk-12.preview.northwind.dev/checkoutSafariChromeFirefoxDesktopPhoneBranchMainSplit

Built from api#219, ds#81, web#482

main

Northwind Checkout

Trail runner, size 42\$89.00

Shipping\$9.00

Tax\$6.50

Total\$104.50

Card number

MM / YY

CVC

Pay \$104.50

chk-12

Northwind Checkout

Trail runner, size 42\$89.00

Shipping\$9.00

Tax\$6.50

Total\$104.50

Apple Pay

Google Pay

or pay with card

Card number

Pay \$104.50

1

MC Maya

Taller than the card field.
Can they match?

Wren is on it

Criteria in this preview

✓ Apple Pay shows on Safari
Seen here and saved as evidence

○ Google Pay shows on Chrome
Switch to Chrome to check

○ Falls back to the card form
Turn wallets off to check

Save as evidence

Pins

1 Match the button heights

Click anywhere on the page to pin a comment.
It lands in the conversation with a screenshot.

SteerNote

Comment to steer Wren. Type @ to mention someone, or select code, a pin, or output to anchor it.

Comment

Terminals are shared, and the agent's is live.

Every worktree gets its own terminals. Watch the agent's session as it runs, open yours beside it, and take over with one key.

Checkout 2.0 / CHK-12 Wallet payments / **CHK-32 Wallet buttons on web checkout**

In progress

TR W

Esc Canvas

ConversationActivityCodePreviewTerminal

webworktree

Wren

Your shell

dev server:3000

dsworktree

storybook:6006

+ New terminal

WrenLive, read-only

Take over⌘T

```
~/wt/web/chk-32 $ pnpm test wallet --run
✓ renders Apple Pay on Safari      38ms
✓ renders Google Pay on Chrome     31ms
✓ falls back to CardFallback       12ms
Tests 3 passed (3)

~/wt/web/chk-32 $ udax evidence attach 3 --from-test wallet
✓ Attached to "Falls back to the card form"

~/wt/web/chk-32 $ pnpm lint src/checkout --fix
4 files checked, 1 fixed

~/wt/web/chk-32 $ git commit -am "Match wallet button height"
```

TRYour shellSame worktree

```
~/wt/web/chk-32 $ git log --oneline -4
3b7f010 Serve Apple Pay domain file in preview
9c2e71b Add wallet buttons and ds variants
a41c9e2 Use sandbox merchant ID in previews
e17b2c0 Merge api#211 address autocomplete

~/wt/web/chk-32 $ curl -sI localhost:3000/.well-known/apple-...
HTTP/1.1 200 OK
content-type: text/plain

~/wt/web/chk-32 $
```

SteerNote

Comment to steer Wren. Type @ to mention someone, or select code, a pin, or output to anchor it.

Comment⌘↵

Approve by evidence.

Reviewers check outcomes first, then read a diff grouped by intent, with the agent’s reasoning beside each change. Risk is summarized, not buried.

● ● ● Review / CHK-31 Tokenize wallet payments

In review

Atlas

Criteria and evidence

✓ Tokens never touch our logs

Scanned logs from 1,200 sandbox requests. 0 matches for token patterns.

View query

✓ Idempotent on retry

Replayed 50 duplicate requests. 1 charge created.

View test run

✓ Sandbox charges pass

Apple Pay and Google Pay, recorded end to end.

Pay

G Pay

Changes, grouped by intent

Accept wallet tokens4 files, +212 -8

Keep tokens out of logs3 files, +18 -11

Request logging printed the whole struct, token included. Log only the wallet type and amount.

41func (s *Service) Tokenize(ctx context.Context, req WalletReq) (Token, error) {

42- log.Info("tokenize", "req", req)

42+ log.Info("tokenize", "wallet", req.Wallet, "amount", req.Amount)

Idempotency keys5 files, +96 -4

Tests2 files, +141

Risk

Blast radiuspayments-service, 2 endpoints

MigrationsNone

Protected pathapi/payments/**

Code review1 of 2, payments team needed

OutcomeTheo accepts, as owner

Approve

Request changes

Take over

Requested changes become a follow-up for Atlas on this issue, not a comment thread.

Same graph, different questions.

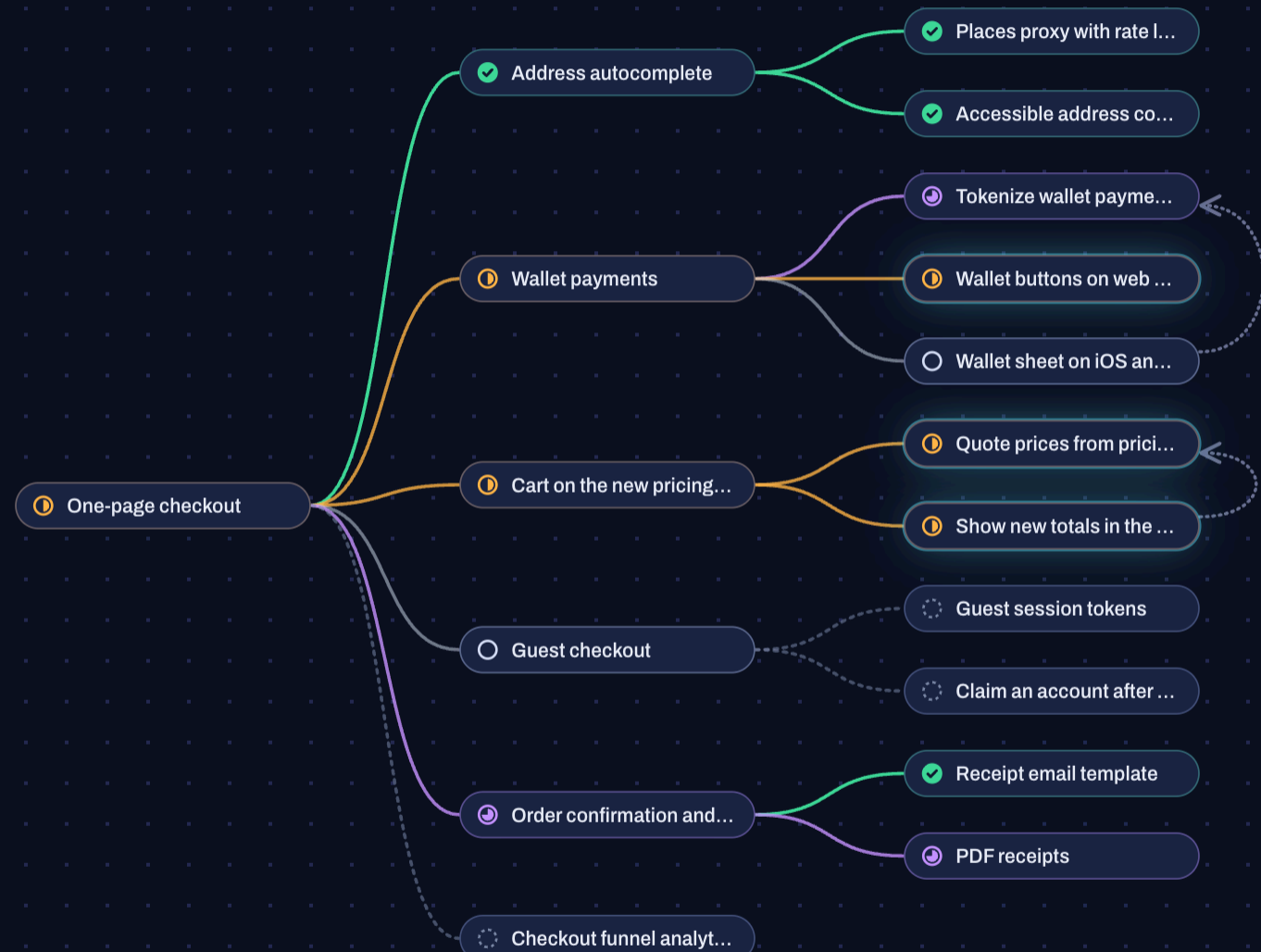
Lenses recolor the canvas without moving anything, so your spatial memory survives. Glyphs keep showing status underneath.

Status Repo Owner Assignee Stuck Risk

Press 1 to 6 to switch.

Where are we?

- Backlog
- To do
- In progress
- In review
- Done



Say it, or press it.

The intent bar turns plain language into graph changes, with a preview before anything is created. Every action also has a key.

Let guests check out without an account. Ship within two weeks.

Preview

Guest checkout

new issue under CHK-1 One-page checkout

Repos

api

web

from similar issues

Target

Two weeks, noted as a constraint for the planner

Next

Plan with Juno as ghosts

Create and plan

Create only

- /split

Ask an agent to propose sub-issues
- /link web#490

Link a PR to the selected issue
- @Wren use the sandbox keys

Steer the agent working here
- ?stuck repo:api

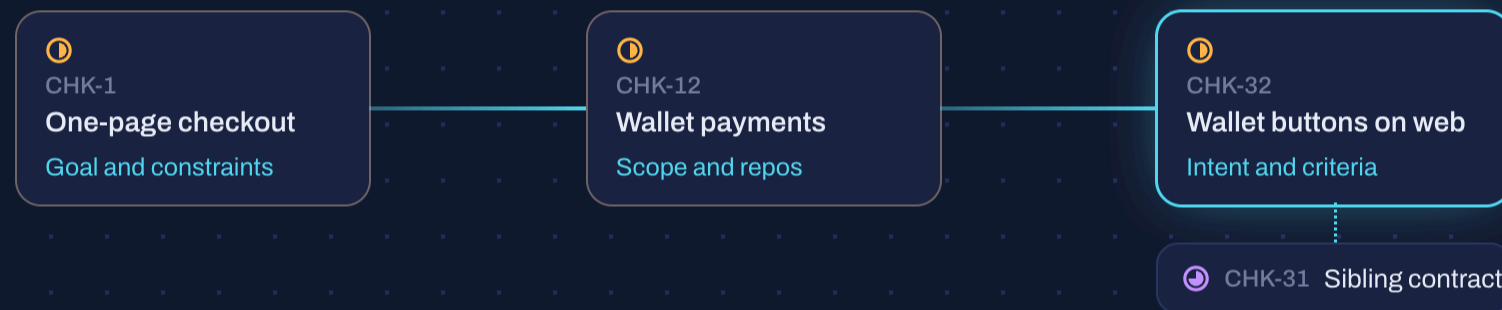
Filter the canvas

Move		Shape	
←	Parent	N	New sub-issue
→	First sub-issue	S	Split with an agent
↑ ↓	Siblings	D	Add dependency
Tab	Next needs-you	L	Link a PR
↵	Enter room	R	Add repository
Esc	Rise one altitude	⇧ 1-5	Set status
Space	Peek		
		See	
		1-6	Switch lens
		⌘K	Jump to anything
		⌘\	Outline view
		⌘Z	Undo, including accepted agent work

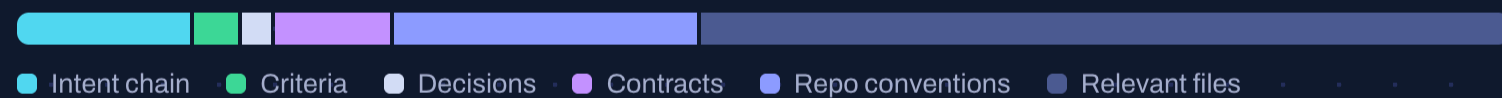
Designed for the agent, too.

The graph is an agent's context. The path from the root to its node is the brief, and every action people click is a `udax` command agents run.

Context cascade for CHK-32



What the agent receives



Every source is listed in the room, so people can see exactly what the agent was told.

Same verbs for people and agents

Split an issue	<code>udax issue propose</code>
Ask a question	<code>udax issue ask</code>
Link a PR	<code>udax pr link</code>
Attach evidence	<code>udax evidence attach</code>
Move status	<code>udax issue status</code>
Flag a collision	<code>udax file claim</code>
Read any conversation	<code>udax issue comment list</code>
Reply to a comment	<code>udax issue comment add</code>

Agents can **read** any issue and its conversation. They **propose** changes, and events and people confirm them. Writes follow the agent's repo scopes, budget, branch autonomy, and the owner it answers to.

One CLI for people and agents.

The same udax commands work in your shell and inside an agent's run. In a run, the CLI already knows which agent it is and what it may change.

TR Theo's shell

```
$ udax setup
✓ Signed in as theo@northwind.dev
✓ Workspace: northwind
✓ Runtime "Theo's MacBook" is online. Found claude, codex.
$ udax issue list --project checkout --status in_progress
KEY      STATUS      ASSIGNEE  TITLE
CHK-32   in_progress Wren      Wallet buttons on web checkout
CHK-40   in_progress Pike      Quote prices from pricing-engine
CHK-41   in_progress Kit       Show new totals in the cart
CHK-17   in_progress none      Cart on the new pricing engine
$ udax issue assign CHK-36 --to atlas --effort max
✓ Assigned CHK-36 to Atlas at Max effort. A run is queued.
$ udax issue comment add CHK-32 --note --content "The fallback works."
✓ Note added. No run started.
$ █
```

W Inside Wren's run / CHK-32, run 2

Scoped token from the runtime

```
$ udax whoami
Wren, agent. Run 2 on CHK-32. Owner: Theo Ramos.
Can write web and ds. Budget left: 26 min, $4.98.
$ udax issue get CHK-32 --output json | jq .criteria
["Apple Pay shows on Safari", "Google Pay shows on Chrome",
  "Falls back to the card form"]
$ udax issue comment list CHK-31 --decisions
DECISION                                BY    WHEN
Tokens expire after 15 minutes.        Theo  Mon
Never log the raw token.                 Theo  Mon
$ udax issue comment add CHK-32 --reply c12 --content-stdin < reply.md
✓ Replied on CHK-32. Theo's request is done.
$ udax issue assign CHK-41 --to wren
✗ Agents can't reassign work. Ask the owner: udax issue ask CHK-41
$ udax login
✗ Not needed inside a run. This run already has a scoped token.
$ █
```

Nouns first, then verbs.

```
udax issue assign CHK-36 --to atlas --effort max
```

noun verb key flags

issue

list, get, create, update,
assign, status, search,
children, ask, propose

issue comment

list, add, resolve

run

list, logs, rerun, cancel, usage

agent

list, get, create, update, copy,
archive

evidence

attach, list

pr

list, link

repo

list, checkout

file

claim, release

graph

show

runtime, daemon

list, status, start, logs

workspace

list, switch

setup, login, config

For people only

Every command follows one grammar, so people and agents can guess the next one. Tables for reading, JSON for scripts, and nothing wakes an agent by surprise.

Keys, not IDs.

`CHK-32` works in every command. `--full-id` prints the UUID when a script needs it.

Tables for people, JSON for machines.

`list` prints a table and `get` prints JSON. `--output` switches either one.

Names resolve.

`--to wren` matches agents and people by name. `--to-id` settles a clash.

Long text goes through stdin.

`--content-stdin` and `--content-file` keep comments and descriptions out of shell quoting.

Side effects are announced.

A command that starts a run says so. `--note` and `--no-start` change things without waking an agent.

Runs get scoped tokens.

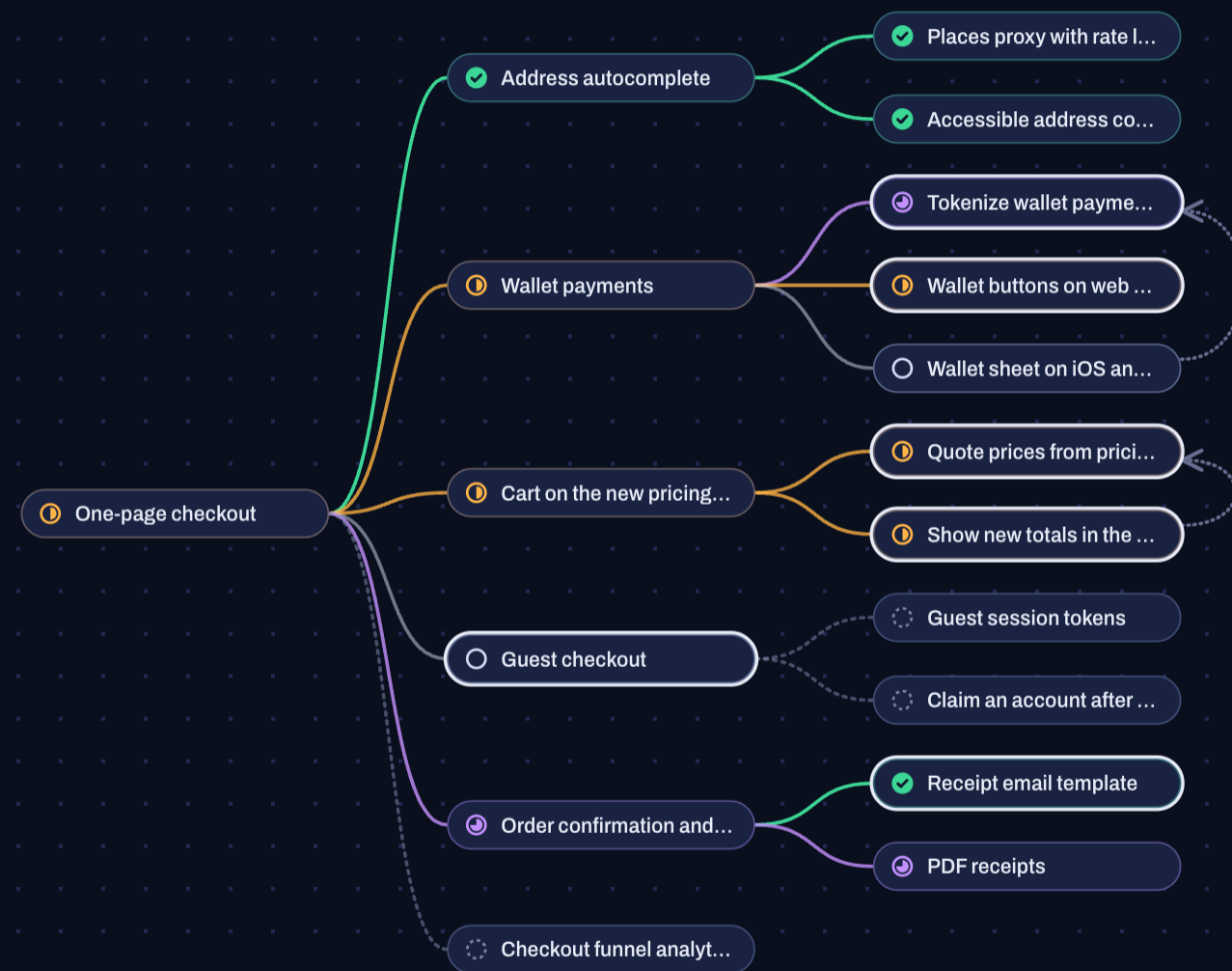
Inside a run, the runtime injects a token for that agent and issue. Human commands like `login` are refused.

Catch up in ten seconds.

When you come back, the canvas replays what changed while you were away, then settles with those nodes outlined. Scrub to any moment.

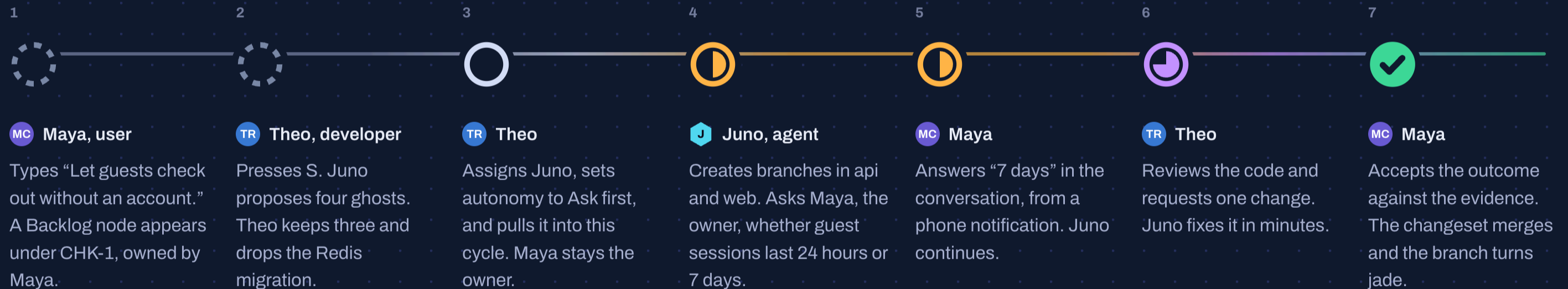
Since you left at 18:00

- 19:12 Atlas opened api#219 for review
- 21:40 Receipt email template merged
- 23:05 Pike started quoting from pricing-engine
- 02:18 Kit opened web#488 as a draft
- 06:30 Juno asked Maya about guest sessions
- 07:55 Wren started wallet buttons



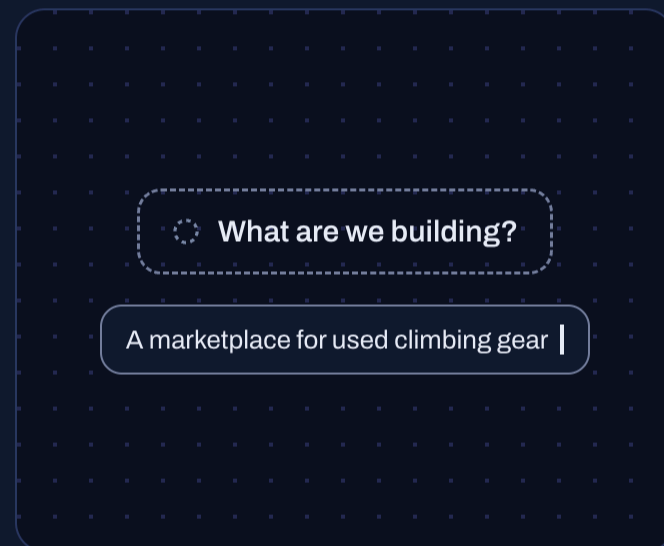
A feature, start to finish.

Guest checkout, from a sentence to merged code. Seven moments, three kinds of users, one node that never changes identity.



Designing the hard moments.

The empty canvas, the huge one, and the ones where something went wrong. Each ends with a clear next step.



Empty project

The first node is a question. Answer it and an agent drafts the first branches as ghosts.



Two thousand issues

Done subtrees fold into a single seed after a week. The outline view takes over when a canvas stops helping.



Agents collide

Flagged at the first overlapping edit, not at a merge. You pick who goes first. The other agent rebases after.



Checks keep failing

Agents retry twice, then escalate with the failing log attached and a proposed fix to approve.

Readable without color. Usable without a mouse.

- **Shape carries status.** Dashed, empty, half, three-quarters, full. Color is a second channel, never the only one.
- **An outline view mirrors the canvas.**  switches to a tree that screen readers announce as a tree, with dependencies read out on focus.
- **Every canvas action has a key,** and focus is always visible on the canvas.
- **Contrast is checked on the canvas ink.** Text meets 4.5:1. Status inks and edges meet 3:1.
- **Reduced motion is respected.** Beacons stop pulsing and become a static ring. Camera moves become cuts.

Canvas



Outline

- | | |
|---|-------------|
| ① CHK-1 One-page checkout | In progress |
| ✓ CHK-13 Address autocomplete | Done |
| ✓ CHK-34 Places proxy with rate limits | Done |
| ✓ CHK-35 Accessible address combobox | Done |
| ① CHK-12 Wallet payments | In progress |
| 🌀 CHK-31 Tokenize wallet payments | In review |
| ① CHK-32 Wallet buttons on web checkout | In progress |
| ○ CHK-33 Wallet sheet on iOS and Android | To do |
| ① CHK-17 Cart on the new pricing engine | In progress |
| ① CHK-40 Quote prices from pricing-engine | In progress |
| ① CHK-41 Show new totals in the cart | In progress |
| ○ CHK-14 Guest checkout | To do |
| 🌀 CHK-36 Guest session tokens | Backlog |
| 🌀 CHK-37 Claim an account after purchase | Backlog |
| 🌀 CHK-15 Order confirmation and receipts | In review |

What we build first, and how we'll know.

Build order

- 1 Graph**
Canvas, issues, the five statuses, sub-issues, owners and assignees, GitHub PR linking, one repo per issue.
- 2 Agents**
Configurable agents, the udx CLI, runs in worktrees, the conversation and activity views, ghosts, needs-you, the autonomy dial.
- 3 Many repos**
Repo tracks, changesets, contract checks, and the code, preview, and terminal views across repos.
- 4 Scale**
Lenses, replay, outline parity, two-way sync with Linear, Jira, and GitHub Issues.

Signals we'll watch

Intent to merged PR

Median time from a new node to its first merge, compared with the team's current tracker.

Reviews decided on evidence

Share of approvals made without expanding every diff group, with no rise in reverts.

Time to answer an agent

How long needs-you beacons wait on owners. If it grows, agents are asking the wrong questions.

Open questions

Past a few hundred visible issues, should big projects open in outline view by default?

Do we own issues, or sync two-way with the tracker a team already uses?

How much autonomy should a new team get on day one?

What counts as evidence for work with no UI, like migrations and infrastructure?

Should agents be able to create and configure other agents, or only people?

✓✓✓✓✓ UDAX

Let's draw the work.

People decide what should exist. Agents build it in the open.
Everyone sees the tree turn green.

